Written Response Questions

1. Fine-tuning strategy for a multiple-class object detection model

After training a multi-class object detection model, fine-tuning is essential for improving generalization and reducing errors on the validation set. I would begin with a detailed error analysis, looking at per-class Average Precision (AP), confusion matrices, and visual inspection of false positives and false negatives. This helps identify whether the issues are due to class imbalance, localization errors, misclassifications, or poor feature learning for specific categories.

Next, I would tune key hyperparameters. Since the learning rate has the strongest effect during fine-tuning, I would lower it so the model can adjust its weights more carefully without losing useful features. If the model is underfitting, I might use a slightly higher learning rate or train for more epochs with early stopping to keep overfitting in check. Batch size also affects training stability, so I would adjust it as needed. I would also experiment with optimizers like AdamW or SGD to improve convergence and generalization.

If certain sizes of objects are detected poorly, I would adjust the anchor boxes, detection head settings, IoU thresholds for positive and negative anchor assignment, NMS thresholds, or even the input image resolution so the model aligns better with the actual object shapes in the dataset. If class imbalance is detected, I might use data augmentation (mixup for small objects, random cropping, or color jittering) to boost underrepresented classes.

I would also consider regularization adjustments, such as dropout, weight decay, or label smoothing, to help reduce overfitting. Additionally, fine-tuning only higher-level layers or selectively unfreezing parts of the backbone can improve performance, especially when training data is limited.

2. Estimate the normal vector at each point in the point cloud

A normal vector at a point on a surface is a vector perpendicular to the surface at that point. In a point cloud, a single point doesn't show the surface, but its nearby points form a small patch. By analyzing these neighbors with PCA, we can find the direction with the least spread, which is perpendicular to the surface.

Algorithm

For each point in the point cloud

- Find nearby points using either a fixed-radius search or by selecting a fixed number of nearest neighbors.
- Compute the average position of the neighboring points.
- Create a covariance matrix by measuring how the neighbors are distributed around their average.
- Perform an eigenvalue decomposition of the covariance matrix.

- Select the eigenvector associated with the smallest eigenvalue as the estimated normal direction.

3. Polygon Simplification Algorithms

The Ramer–Douglas–Peucker (RDP) algorithm and the Visvalingam–Whyatt (VW) algorithm are the two widely used algorithms for polygon simplification. Both methods aim to reduce the number of points in a polygonal chain while preserving its overall geometric characteristics.

3.1. Ramer–Douglas–Peucker (RDP) Algorithm

Main Idea
The RDP algorithm is based on a divide-and-conquer strategy that measures the perpendicular distance of points from a line segment. The algorithm recursively keeps points that deviate significantly from the straight line joining two endpoints, thereby preserving the essential shape of the polygon.

How it decides which points to keep or remove

i. Start with the first and last points of the polygonal chain.
ii. Draw a straight line between these two points.
iii. Find the point with the maximum perpendicular distance from this line.
iv. If this maximum distance is greater than a user-defined tolerance ε:
    - Keep the point.
    - Recursively apply the algorithm to the two line segments created.
v. If the distance is less than ε:
    - Discard all intermediate points.

Pseudocode

```
def douglas_peucker(points, epsilon):
  if len(points) <= 2:
    return points

  first, last = points[0], points[-1]
  max_dist, index = 0, 0
  for i, p in enumerate(points[1:-1], start=1):
    dist = perpendicular_distance(p, first, last)
    if dist > max_dist:
      max_dist, index = dist, i

  if max_dist > epsilon:
    left = douglas_peucker(points[:index+1], epsilon)
    right = douglas_peucker(points[index:], epsilon)
```

```
    return left[:-1] + right
  else:
    return [first, last]
```

Advantages

- Preserves the most significant deviations in the shape.
- Easy to control the level of simplification with a tolerance parameter.

Drawbacks

- Sensitive to the order of points in the polygon.
- Can remove small features that may be important if the tolerance is too large.

3.2. Zhou-Jones Algorithm

Main Idea

The Zhou-Jones algorithm is a method for cartographic line generalization/simplification that uses weighted effective areas to determine and remove the least cartographically significant vertices of a line. It is a modification of the Visvalingam-Whyatt algorithm.

How it decides which points to keep or remove

i. Compute Effective Area
   - For each vertex, form a triangle with its two neighboring points.
   - Calculate the triangle's area to measure the vertex's importance.

ii. Apply Weight Factors
   - Modify each effective area using weights based on metrics such as flatness, skewness, and convexity.

iii. Remove the Least Important Vertex
   - Identify the vertex with the smallest weighted effective area.
   - Remove that vertex and recalculate areas for its neighboring vertices.

iv. Continue removing the least important vertices until the desired simplification level is reached.

Advantages

- Allows precise control over the number of points in the output.
- Produces smooth and visually pleasing simplifications.

Drawbacks

- Computationally expensive.
- Greedy local decisions may not produce a globally optimal simplification.
- Can remove small but important features before larger, less meaningful ones.